

✓ Lab 13: Neural Networks Handwritten Numbers Reader

Goal: Build an AI that can look at messy handwriting and tell us what number it is (0, 1, 2... 9).

Lab Objectives

By the end of this lab, you will be able to:

1. **Visualize Data:** See how computers view handwritten numbers as pixel grids.
2. **Process Images:** Prepare image data for Artificial Intelligence.
3. **Build a Classifier:** Create a Neural Network that identifies patterns (loops vs lines).
4. **Test Accuracy:** Check if your AI can read handwriting better than a human!

Introduction to the Data: MNIST

We are using the **MNIST** dataset (Modified National Institute of Standards and Technology).

- **Content: 70,000** images of handwritten digits (0-9).
 - **Source:** These were written by high school students and census bureau employees.
 - **Challenge:** Everyone writes differently! Some 7s look like 1s. Some 0s look like 6s.
-

Part 1: Import Libraries

```
# 1. Import 'numpy' (Math tool)
import numpy as np

# 2. Import 'tensorflow' (AI tool)
import tensorflow as tf

# 3. Import 'keras' (Easy AI builder)
from tensorflow import keras
```



```
[ 0 0 0 0 0 0 0 0 0 0 0 0 3 18 18 18 126 136
175 26 166 255 247 127 0 0 0 0]
[ 0 0 0 0 0 0 0 0 30 36 94 154 170 253 253 253 253 253
225 172 253 242 195 64 0 0 0 0]
[ 0 0 0 0 0 0 0 49 238 253 253 253 253 253 253 253 253 251
93 82 82 56 39 0 0 0 0 0]
[ 0 0 0 0 0 0 0 18 219 253 253 253 253 253 198 182 247 241
0 0 0 0 0 0 0 0 0 0]
[ 0 0 0 0 0 0 0 0 80 156 107 253 253 205 11 0 43 154
0 0 0 0 0 0 0 0 0 0]
[ 0 0 0 0 0 0 0 0 0 14 1 154 253 90 0 0 0 0
0 0 0 0 0 0 0 0 0 0]
[ 0 0 0 0 0 0 0 0 0 0 0 139 253 190 2 0 0 0
0 0 0 0 0 0 0 0 0 0]
[ 0 0 0 0 0 0 0 0 0 0 0 11 190 253 70 0 0 0
0 0 0 0 0 0 0 0 0 0]
[ 0 0 0 0 0 0 0 0 0 0 0 0 35 241 225 160 108 1
0 0 0 0 0 0 0 0 0 0]
[ 0 0 0 0 0 0 0 0 0 0 0 0 0 81 240 253 253 119
25 0 0 0 0 0 0 0 0 0]
[ 0 0 0 0 0 0 0 0 0 0 0 0 0 0 45 186 253 253
150 27 0 0 0 0 0 0 0 0]
[ 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 16 93 252
253 187 0 0 0 0 0 0 0 0]
[ 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 249
253 249 64 0 0 0 0 0 0 0]
[ 0 0 0 0 0 0 0 0 0 0 0 0 0 0 46 130 183 253
253 207 2 0 0 0 0 0 0 0]
[ 0 0 0 0 0 0 0 0 0 0 0 0 0 39 148 229 253 253 253
250 182 0 0 0 0 0 0 0 0]
[ 0 0 0 0 0 0 0 0 0 0 24 114 221 253 253 253 253 201
78 0 0 0 0 0 0 0 0 0]
[ 0 0 0 0 0 0 0 0 23 66 213 253 253 253 253 198 81 2
0 0 0 0 0 0 0 0]
[ 0 0 0 0 0 0 18 171 219 253 253 253 253 195 80 9 0 0
0 0 0 0 0 0 0 0]
[ 0 0 0 0 55 172 226 253 253 253 253 244 133 11 0 0 0 0
0 0 0 0 0 0 0 0]
[ 0 0 0 0 136 253 253 253 212 135 132 16 0 0 0 0 0 0
0 0 0 0 0 0 0 0]
[ 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0]
[ 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0]
```

✓ Part 3: The "0 to 1" Rule

Just like before, the pixels are 0 to 255. We divide by 255 to make them 0 to 1. This helps the AI learn faster.

```
# Shrink the numbers to be between 0 and 1
train_images = train_images / 255.0
test_images = test_images / 255.0

print("Normalization complete")
```

```
Normalization complete
```

✓ Part 4: Compiling Model

We need a brain that can find loops (like in 0, 6, 8, 9) and straight lines (like in 1, 7, 4).

[Image of neural network for digit recognition]

1. **Flatten:** Unroll the 28x28 square image into a long line (784 pixels).
2. **Dense (256):** The thinking layer.
3. **Dense (10):** The answer layer (0, 1, 2, 3, 4, 5, 6, 7, 8, 9).

```
# Step 1: Create the empty brain
model = keras.Sequential()

# Step 2: Unroll the square image
model.add(keras.layers.Flatten(input_shape=(28, 28)))

# Step 3: Thinking layer
model.add(keras.layers.Dense(256, activation='relu'))

# Step 4: Output layer (10 neurons for digits 0-9)
model.add(keras.layers.Dense(10, activation='softmax'))
```

```
# Step 5: Setup learning rules
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

print("Model Compiled")
```

Model Compiled

✓ Part 5: Train the Model

We will show the AI the 60,000 study images **10 times** (Epochs).

```
# Teach the AI!
model.fit(train_images, train_labels, epochs=10)
```

```
Epoch 1/10
1875/1875 ————— 14s 7ms/step - accuracy: 0.6261 - loss: 1.4692
Epoch 2/10
1875/1875 ————— 11s 6ms/step - accuracy: 0.8852 - loss: 0.4283
Epoch 3/10
1875/1875 ————— 21s 6ms/step - accuracy: 0.9028 - loss: 0.3449
Epoch 4/10
1875/1875 ————— 11s 6ms/step - accuracy: 0.9114 - loss: 0.3101
Epoch 5/10
1875/1875 ————— 11s 6ms/step - accuracy: 0.9164 - loss: 0.2883
Epoch 6/10
1875/1875 ————— 10s 6ms/step - accuracy: 0.9201 - loss: 0.2776
Epoch 7/10
1875/1875 ————— 10s 6ms/step - accuracy: 0.9261 - loss: 0.2545
Epoch 8/10
1875/1875 ————— 11s 6ms/step - accuracy: 0.9264 - loss: 0.2503
Epoch 9/10
1875/1875 ————— 11s 6ms/step - accuracy: 0.9351 - loss: 0.2256
Epoch 10/10
1875/1875 ————— 11s 6ms/step - accuracy: 0.9362 - loss: 0.2173
<keras.src.callbacks.history.History at 0x78ff35d55f40>
```

✓ Part 6: Prediction

Pick a random image ID. The computer will draw the handwritten number, and the AI will try to read it.

```
# --- STUDENT AREA: CHANGE THIS NUMBER (0 to 9999) ---
image_id = 1000
# -----

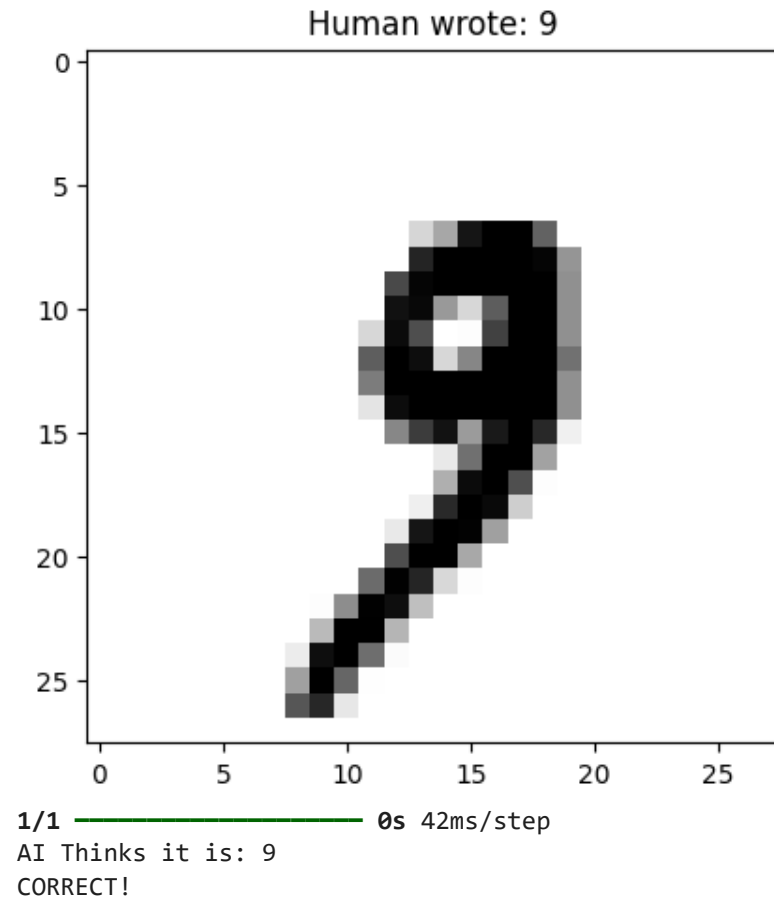
# 1. Get the image and the real answer
picture = test_images[image_id]
real_answer = test_labels[image_id]

# 2. Draw the handwriting on screen
plt.imshow(picture, cmap=plt.cm.binary)
plt.title(f"Human wrote: {real_answer}")
plt.show()

# 3. Ask AI to guess
prediction = model.predict(np.array([picture]))
predicted_number = np.argmax(prediction[0])

print(f"AI Thinks it is: {predicted_number}")

if predicted_number == real_answer:
    print("CORRECT!")
else:
    print("WRONG!")
```



✓ Part 7: Lab Submission

Student Name: _____

Task 1: The Detective Change the `image_id` until you find a very messy number that looks ugly to you.

- Image ID: _____
- Did the AI still read it correctly? (Yes/No)

Task 2: The Confusion Find an image where the AI made a mistake (or almost made one).

- Which number was it really?
- What number did the AI think it was?
- Why do these two numbers look similar? (e.g., "A sloppy 1 looks like a 7")

Task 3: Brain Power In Part 4, we used 256 neurons in the thinking layer. If we changed this to only **5 and 128 neurons**, do you think the AI would get smarter or dumber?

Start coding or [generate](#) with AI.